

Python Boolean Logic & Operators

Complete Revision Notes & Reference Guide

1. Boolean Values

Python has two primitive Boolean values representing truth states:

```
True
False

# Check their types:
type(True)    # <class 'bool'>
type(False)   # <class 'bool'>
```

2. Truthy and Falsy Values

Python implicitly evaluates any object as either **Truthy** or **Falsy**. You can check any value's truthiness using the built-in `bool()` function.

Falsy Values	Truthy Values
<code>False</code> , <code>None</code>	<code>True</code>
<code>0</code> , <code>0.0</code> (Numeric zero)	Non-zero numbers (<code>1</code> , <code>-1</code> , <code>10.5</code>)
<code>""</code> (Empty string)	Non-empty strings (<code>"hello"</code> , <code>"0"</code> , <code>" "</code>)
<code>[]</code> , <code>()</code> (Empty list / tuple)	Non-empty lists / tuples (<code>[1]</code> , <code>(1,)</code>)
<code>{}</code> , <code>set()</code> (Empty dict / set)	Non-empty dicts / sets (<code>{"a": 1}</code> , <code>{1}</code>)

```
# Falsy examples:
bool(False) # False
bool(0)     # False
bool("")    # False
bool([])    # False

# Truthy examples:
bool(True)  # True
bool(1)     # True
bool("hello") # True
bool([1])   # True
```

3. Empty vs Non-Empty Containers

- An empty container is generally **Falsy**.
- A non-empty container is generally **Truthy**.

Key Distinction to Remember

`""` is an empty string (Falsy), whereas `"0"` is a string containing one character (Truthy).
Similarly, `[]` is empty (Falsy), but `[0]` or `[False]` is a non-empty container with 1 item (Truthy).

```
bool("")          # False (empty string)
bool("0")         # True (string with one character)

bool([])          # False (empty list)
bool([False])     # True (list contains 1 item)
bool([0])         # True (list contains 1 item)
bool([""])        # True (list contains 1 item)
```

4. The `and` Operator

Main Rule

`and` evaluates left to right.

- Returns the **first Falsy value** encountered.
- If there is no Falsy value, it returns the **last value**.

```
# Basic Pattern: A and B
True and 1  # 1  (True is Truthy -> continue to last value)
0 and True  # 0  (0 is Falsy -> STOP immediately)
1 and 0     # 0  (1 is Truthy -> continue, 0 is Falsy -> STOP)

# Multiple Values:
10 and 20 and 30      # 30 (All Truthy -> returns last value)
10 and 20 and 0 and 50 # 0  (Stops at 0)
10 and 20 and "" and 50 # "" (Stops at "")
```

5. The `or` Operator

Main Rule

`or` evaluates left to right.

- Returns the **first Truthy value** encountered.
- If there is no Truthy value, it returns the **last value**.

```
# Basic Pattern: A or B
False or 1  # 1  (False is Falsy -> continue to 1)
1 or 0      # 1  (1 is Truthy -> STOP)
0 or 100     # 100 (0 is Falsy -> continue to 100)
False or 0   # 0  (Both Falsy -> returns last value)

# Multiple Values:
0 or "" or 50 # 50 (0 Falsy -> "" Falsy -> 50 Truthy -> STOP)
1 or 20 or 30 # 1  (First value is Truthy -> STOP)
```

6. Short-Circuit Evaluation

Python stops evaluating an expression as soon as the result is determined:

- **and Short-Circuit:** Stops at the first `False` value.
- **or Short-Circuit:** Stops at the first `Truthy` value.

```
0 and False and 1  # 0 (Stops at 0; False and 1 are never evaluated)
10 or 20 or 30      # 10 (Stops at 10; 20 and 30 are never evaluated)
```

7. The `not` Operator

`not` reverses the truth value and **always returns a strict Boolean** (`True` or `False`).

```
not True    # False
not False   # True

not 0        # True  (0 is Falsy)
not 10       # False (10 is Truthy)
not ""       # True  (" " is Falsy)
not "0"      # False ("0" is Truthy)
not []       # True  ([] is Falsy)
not [0]      # False ([0] is Truthy)
```

Crucial Difference: and/or vs not

`and` and `or` return the **actual operand value** (e.g. `False or 10` returns `10`).
`not` **always returns a Boolean** (`True` or `False`).

8. Operator Precedence

When combined without parentheses, Python evaluates operators in this exact hierarchy:

```
( ) (Parentheses)
  ↓
not
  ↓
and
  ↓
or
```

9. Evaluation Examples Step-by-Step

Example A: Precedence of `not`, `and`, `or`

Expression: `not 0 and 10 or 20`

1. **Step 1 (`not`):** `not 0` → `True`
2. **Step 2 (`and`):** `True and 10` → `10`
3. **Step 3 (`or`):** `10 or 20` → `10`

Result: `10`

Example B: Using Parentheses

Expression: `(False or 10) and 5`

1. Step 1 (`()`): `(False or 10) → 10`
2. Step 2 (`and`): `10 and 5 → 5`

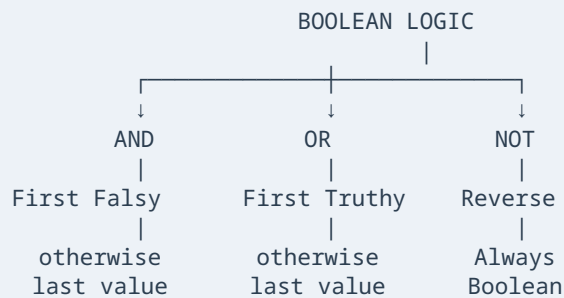
Result: `5`

10. Comparison Operators Integration

Comparison operators (`>`, `<`, `>=`, `<=`, `==`, `!=`) return strict Booleans and have higher precedence than Boolean operators.

```
10 > 5 and 20 > 10 # True and True --> True
10 < 5 or 20 > 10  # False or True  --> True
not (10 > 5)        # not (True)    --> False
```

11. Master Summary Chart



One-Line Rules to Memorize

1. **and** → returns first Falsy, otherwise last value.
2. **or** → returns first Truthy, otherwise last value.
3. **not** → reverses truth value, always returns explicit Boolean.
4. Evaluation order: `()` → **not** → **and** → **or**.